```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>OCS Completeness Checker - ULTRA MAXXX</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'Monaco', 'Courier New', monospace;
            background: #0a0e27;
            color: #e0e0e0;
            padding: 12px;
            width: 580px;
            max-height: 900px;
            overflow-y: auto;
        }

        .header {
            border-bottom: 2px solid #2a5d9f;
            padding-bottom: 10px;
            margin-bottom: 12px;
        }

        .header h2 {
            font-size: 13px;
            color: #4a9eff;
            text-transform: uppercase;
            letter-spacing: 2px;
            margin-bottom: 4px;
        }

        .header p {
            font-size: 10px;
            color: #666;
            text-transform: uppercase;
        }

        .controls {
            display: flex;
```

```
    gap: 6px;
    margin-bottom: 12px;
    flex-wrap: wrap;
}

button {
    padding: 6px 10px;
    background: #1a2a3a;
    border: 1px solid #2a5d9f;
    color: #4a9eff;
    cursor: pointer;
    font-size: 11px;
    font-weight: bold;
    text-transform: uppercase;
    transition: all 0.2s;
    flex: 1;
    min-width: 80px;
}

button:hover {
    background: #2a3a4a;
    border-color: #6ab5ff;
}

button.active {
    background: #4a9eff;
    color: #000;
    border-color: #4a9eff;
}

.layer-btn.active {
    background: #ff6b6b;
    color: #fff;
    border-color: #ff6b6b;
}

textarea {
    width: 100%;
    height: 80px;
    padding: 8px;
    background: #0f1419;
    border: 1px solid #2a5d9f;
    color: #e0e0e0;
    font-family: 'Monaco', 'Courier New', monospace;
    font-size: 11px;
    resize: vertical;
```

```
    margin-bottom: 12px;
}

.layer-selector {
    border-top: 1px solid #2a5d9f;
    padding-top: 8px;
    margin-bottom: 12px;
    display: flex;
    gap: 4px;
    flex-wrap: wrap;
    font-size: 10px;
}

.layer-selector span {
    color: #666;
    text-transform: uppercase;
    align-self: center;
}

.findings {
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.finding {
    border-left: 3px solid #ff6b6b;
    background: #0f1419;
    padding: 8px;
    border-radius: 2px;
}

.finding.warning {
    border-left-color: #ffd93d;
}

.finding.info {
    border-left-color: #4a9eff;
}

.finding.critical {
    border-left-color: #ff4444;
    background: #1a0f0f;
}

.finding.success {
```

```
    border-left-color: #6bff6b;
    background: #0f1a0f;
}

.finding-type {
    font-size: 10px;
    text-transform: uppercase;
    font-weight: bold;
    margin-bottom: 3px;
    color: #ff6b6b;
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
}

.finding.warning .finding-type {
    color: #ffd93d;
}

.finding.info .finding-type {
    color: #4a9eff;
}

.finding.critical .finding-type {
    color: #ff4444;
}

.finding.success .finding-type {
    color: #6bff6b;
}

.severity {
    font-size: 9px;
    padding: 2px 4px;
    background: rgba(255, 107, 107, 0.2);
    border-radius: 2px;
}

.finding.warning .severity {
    background: rgba(255, 217, 61, 0.2);
}

.finding.info .severity {
    background: rgba(74, 158, 255, 0.2);
}
```

```
.finding-text {
    font-size: 11px;
    line-height: 1.4;
    margin-bottom: 5px;
    color: #d0d0d0;
}

.finding-detail {
    font-size: 10px;
    color: #888;
    margin: 3px 0;
    padding-left: 6px;
    border-left: 1px solid #333;
}

.finding-mechanism {
    font-size: 10px;
    color: #aaa;
    background: rgba(74, 158, 255, 0.1);
    padding: 4px 6px;
    margin: 4px 0;
    border-radius: 2px;
}

.finding-observability {
    font-size: 9px;
    color: #999;
    background: rgba(153, 102, 255, 0.1);
    padding: 3px 5px;
    margin: 3px 0;
    border-radius: 2px;
}

.finding-propagation {
    font-size: 9px;
    color: #999;
    margin-top: 4px;
    padding: 3px 0;
}

.empty-state {
    color: #555;
    text-align: center;
    padding: 24px;
    font-size: 11px;
```

```
}

.badge {
    display: inline-block;
    background: rgba(74, 158, 255, 0.2);
    padding: 2px 5px;
    border-radius: 2px;
    font-size: 9px;
    margin: 3px 3px 0 0;
    color: #4a9eff;
}

.badge.layer {
    background: rgba(255, 107, 107, 0.2);
    color: #ff6b6b;
}

.badge.metrological {
    background: rgba(153, 102, 255, 0.2);
    color: #9966ff;
}

.dependency-map {
    font-size: 9px;
    color: #888;
    margin-top: 3px;
    padding: 3px 4px;
    background: rgba(0, 0, 0, 0.3);
    border-left: 2px solid #444;
}

.stats {
    background: #0f1419;
    border: 1px solid #2a5d9f;
    padding: 8px;
    margin-bottom: 12px;
    border-radius: 2px;
    font-size: 10px;
    display: grid;
    grid-template-columns: 1fr 1fr 1fr 1fr;
    gap: 8px;
}

.stat-item {
    text-align: center;
}
```

```
.stat-value {
    font-weight: bold;
    color: #4a9eff;
    font-size: 14px;
}

.stat-label {
    color: #666;
    text-transform: uppercase;
}

.bidirectional-check {
    background: #0f1419;
    border: 1px solid #2a5d9f;
    padding: 8px;
    margin-bottom: 12px;
    border-radius: 2px;
    font-size: 9px;
}

.bidirectional-check h4 {
    color: #4a9eff;
    font-size: 10px;
    margin-bottom: 4px;
    text-transform: uppercase;
}

.bi-item {
    margin: 3px 0;
    padding: 3px 4px;
    background: rgba(74, 158, 255, 0.05);
    border-left: 2px solid #2a5d9f;
}

.synonym-group {
    background: #1a0f0f;
    border: 1px solid #ff6b6b;
    padding: 6px;
    margin: 4px 0;
    border-radius: 2px;
    font-size: 9px;
}

.synonym-group h5 {
    color: #ff6b6b;
```

```
            font-size: 9px;
            margin-bottom: 2px;
        }

        .synonym-items {
            color: #999;
            margin: 2px 0;
        }
    </style>
</head>
<body>
    <div class="header">
        <h2>⚙️ OCS Completeness Checker</h2>
        <p>Ultra MAXXX Edition - Full Epistemology Verification</p>
    </div>

    <div class="controls">
        <button id="checkBtn" class="active">Analyze</button>
        <button id="refineBtn">Refine ($D_1$-$D_5$)</button>
        <button id="clearBtn">Clear</button>
    </div>

    <div class="layer-selector">
        <span>Analysis Layers:</span>
        <button class="layer-btn active" data-layer="A0">$A_0$</button>
        <button class="layer-btn active" data-layer="phaseA">Phase A</button>
        <button class="layer-btn active" data-layer="phaseB">Phase B</button>
        <button class="layer-btn active" data-layer="phaseC">Phase C</button>
        <button class="layer-btn active" data-layer="metrological">Metrological</button>
        <button class="layer-btn active" data-layer="synonym">Anti-Synonym</button>
        <button class="layer-btn active" data-layer="bidirectional">Bidirectional</button>
    </div>

    <textarea id="textInput" placeholder="Paste text here to check for epistemological
completeness..."></textarea>

    <div class="stats" id="stats"></div>

    <div id="bidirectionalSection"></div>

    <div class="findings" id="findings">
        <div class="empty-state">Enter text and click "Analyze" to begin</div>
    </div>

    <script>
```

```
    const ocsPatterns = [
        // A₀ - ASSUMPTION VACUUM SCAN
        {
            layer: "A0",
            name: "Unstated Assumption",
            regex: /\b(obviously|clearly|naturally|of course|as expected|it is known that|
everyone knows|as we all know|needless to say|it goes without saying)\b/gi,
            type: "critical",
            severity: 9,
            message: (m) => `Assumption flagged: "${m}" — state the underlying premise
explicitly`,
            propagation: "All dependent claims inherit this uncertainty",
            metrological: { confidence: "Low", observability: "Non-observable",
measurement: "Not determinable" }
        },
        {
            layer: "A0",
            name: "Latent Condition Unexamined",
            regex: /\b(in normal circumstances|typically|generally|usually|in most cases|
standard conditions)\b/gi,
            type: "warning",
            severity: 7,
            message: (m) => `Boundary condition unspecified: "${m}" — what are the
exceptions?`,
            propagation: "Applicability scope is undefined",
            metrological: { confidence: "Moderate", observability: "Indirect", measurement:
"Conditionally determinable" }
        },
        {
            layer: "A0",
            name: "Temporal Degradation Unaddressed",
            regex: /\b(continues|persists|remains|stays|keeps|holds|maintains|endures)
\b\s+(the same|stable|consistent|unchanged)/gi,
            type: "info",
            severity: 5,
            message: (m) => `No decay vector specified: ${m} — identify temporal
boundaries`,
            propagation: "Durability assumptions cascade forward",
            metrological: { confidence: "Moderate", observability: "Indirect", measurement:
"Conditionally determinable" }
        },

        // PHASE A - SYSTEM EXTRACTION
        {
            layer: "phaseA",
            name: "Implicit Mechanism",
```

```
            regex: /\b(works|operates|functions|drives|ensures|handles|manages|
processes|executes|enables|facilitates|performs|controls|creates|produces|makes|
causes)\b/gi,
            type: "critical",
            severity: 9,
            message: (m) => `Mechanism unspecified: "${m}" — HOW does this occur?`,
            mechanism: "Verb requires mechanical decomposition",
            propagation: "All dependents lack causal grounding",
            metrological: { confidence: "Low", observability: "Non-observable",
measurement: "Not determinable" }
        },
        {
            layer: "phaseA",
            name: "Missing Component Extraction",
            regex: /\b(system|process|mechanism|structure|network|framework|
architecture|apparatus)\b(?!\s+(?:has|contains|includes|consists of|comprises))/gi,
            type: "warning",
            severity: 6,
            message: (m) => `Component not decomposed: "${m}" — extract
subcomponents`,
            propagation: "Hierarchical structure is flat",
            metrological: { confidence: "Moderate", observability: "Indirect", measurement:
"Conditionally determinable" }
        },
        {
            layer: "phaseA",
            name: "Unresolved Relationship",
            regex: /([A-Z][a-z]+)\s+(?:and|with|plus)\s+([A-Z][a-z]+)\s+(?:interact|relate|
connect|link)/gi,
            type: "warning",
            severity: 6,
            message: (m) => `Relational unit incomplete: ${m} — specify dependency
direction`,
            propagation: "Relationship cardinality unknown",
            metrological: { confidence: "Low", observability: "Indirect", measurement:
"Conditionally determinable" }
        },

        // PHASE B - DETERMINISTIC CLASSIFICATION
        {
            layer: "phaseB",
            name: "Weak Inference Presented as Fact",
            regex: /\b(seems|appears|might|could|may|possibly|arguably|suggest|hint at|
intimate|imply|propose)\b\s+([^.!?]{20,60})/gi,
            type: "warning",
            severity: 7,
```

```
            message: (m) => `Inference not classified: ${m} — mark confidence level`,
            propagation: "Fact/inference boundary compromised",
            metrological: { confidence: "Low", observability: "Indirect", measurement:
"Conditionally determinable" }
        },
        {
            layer: "phaseB",
            name: "Uncertainty Not Propagated",
            regex: /\b(unknown|uncertain|unclear|ambiguous|not determined|to be
determined|TBD)\b/gi,
            type: "info",
            severity: 6,
            message: (m) => `Unresolved variable: "${m}" — which downstream units
inherit this?`,
            propagation: "Dependent claims are tainted",
            metrological: { confidence: "Low", observability: "Non-observable",
measurement: "Not determinable" }
        },
        {
            layer: "phaseB",
            name: "Dependency Not Marked",
            regex: /([A-Z][^.!?]*?)\s+(requires|depends on|needs|assumes|presumes|
relies on)\s+([^.!?]+[.!?])/gi,
            type: "info",
            severity: 5,
            message: (m) => `Dependency found but not structured: verify prerequisite
satisfaction`,
            propagation: "Dependency chain may be severed",
            metrological: { confidence: "Moderate", observability: "Direct", measurement:
"Determinable from source" }
        },

        // PHASE C - OUTPUT ASSEMBLY
        {
            layer: "phaseC",
            name: "Compression Corruption Detected",
            regex: /\b(etc|and so on|things like that|you know|kind of|sort of|like|basically|
essentially|ultimately|really)\b/gi,
            type: "warning",
            severity: 7,
            message: (m) => `Compression detected: "${m}" — substitute with explicit
structure`,
            propagation: "Information loss in assembly",
            metrological: { confidence: "High", observability: "Direct", measurement:
"Determinable from source" }
        },
```

```
        {
            layer: "phaseC",
            name: "Surface Fluency Over Structure",
            regex: /([.!?]\s*[A-Z][a-z]{1,3}\s+[a-z]+\s+[a-z]+\s+[a-z]+\.){3,}/g,
            type: "info",
            severity: 4,
            message: (m) => `Short sentences suggest primacy of fluency — verify
structural necessity`,
            propagation: "May obscure dependencies",
            metrological: { confidence: "Moderate", observability: "Direct", measurement:
"Determinable from source" }
        },

        // CROSS-LAYER CRITICAL
        {
            layer: "phaseA",
            name: "Missing Causation Chain",
            regex: /\b(leads to|results in|causes|produces|creates|generates|effects|
affects|impacts)\s+([a-z]+)/gi,
            type: "critical",
            severity: 8,
            message: (m) => `Causal chain incomplete: ${m} — specify mechanism
between cause and effect`,
            propagation: "Effect claims are ungrounded",
            metrological: { confidence: "Low", observability: "Indirect", measurement:
"Conditionally determinable" }
        },
        {
            layer: "phaseB",
            name: "Vague Boundary Condition",
            regex: /\b(sometimes|often|rarely|occasionally|in some cases|in certain
contexts|under specific conditions)\b/gi,
            type: "warning",
            severity: 7,
            message: (m) => `Boundary undefined: "${m}" — specify exact conditions`,
            propagation: "Applicability scope is ambiguous",
            metrological: { confidence: "Low", observability: "Indirect", measurement:
"Conditionally determinable" }
        },
        {
            layer: "phaseA",
            name: "Unquantified Universal Claim",
            regex: /\b(all|none|every|any|always|never|everyone|no one)\b/gi,
            type: "critical",
            severity: 8,
```

```
            message: (m) => `Universal claim unqualified: "${m}" — specify exceptions or
scope`,
            propagation: "Boundary conditions are absent",
            metrological: { confidence: "Low", observability: "Direct", measurement: "Not
determinable" }
        },
        {
            layer: "metrological",
            name: "Observation Confidence Unmarked",
            regex: /\b(observed|found|discovered|noted|reported|claimed|stated|asserted)
\b/gi,
            type: "info",
            severity: 5,
            message: (m) => `Observability unmarked: "${m}" — classify as direct/
indirect/inferred`,
            propagation: "Detection confidence is unknown",
            metrological: { confidence: "Moderate", observability: "Direct", measurement:
"Determinable from source" }
        },
    ];

    const activeLayers = {
        A0: true,
        phaseA: true,
        phaseB: true,
        phaseC: true,
        metrological: true,
        synonym: true,
        bidirectional: true
    };

    let previousAnalysis = null;

    function detectSynonymCollapse(text) {
        const synonymGroups = [
            { group: "System Terms", items: ["system", "apparatus", "mechanism",
"structure", "network"], differences: "Identity, function, scale, hierarchy level" },
            { group: "Process Terms", items: ["process", "procedure", "method",
"mechanism", "operation"], differences: "Temporality, intentionality, iteration" },
            { group: "Relationship Terms", items: ["relates", "interacts", "connects", "links",
"associates"], differences: "Direction, causation, strength, dependency" },
            { group: "Knowledge Terms", items: ["know", "understand", "believe", "think",
"assume"], differences: "Confidence level, epistemological grounding, justification" }
        ];

        const collapses = [];
```

```
        synonymGroups.forEach(group => {
            const foundItems = group.items.filter(item => new RegExp(`\\b${item}\\b`,
'gi').test(text));
            if (foundItems.length > 1) {
                collapses.push({
                    group: group.group,
                    items: foundItems,
                    differences: group.differences
                });
            }
        });

        return collapses;
    }

    function bidirectionalValidation(text, findings) {
        // Forward: text → extracted findings
        const extractedCount = findings.length;

        // Reverse: reconstruct from findings what would be lost
        const sentences = text.split(/[.!?]+/).filter(s => s.trim()).length;
        const words = text.split(/\s+/).length;
        const components = (text.match(/\b(system|process|mechanism|component|unit|
layer|phase)\b/gi) || []).length;
        const mechanisms = (text.match(/\b(works|operates|functions|drives|ensures|
handles|manages|processes|executes|enables|facilitates|performs|controls|creates|
produces|makes|causes)\b/gi) || []).length;
        const uncertainties = (text.match(/\b(unknown|uncertain|unclear|ambiguous|
possibly|might|could|may)\b/gi) || []).length;

        const reconstructionQuality = {
            sentencePreservation: sentences,
            wordDensity: words,
            componentExtraction: components,
            mechanismPreservation: mechanisms,
            uncertaintyMarking: uncertainties,
            compressionRisk: extractedCount > 0 ? ((sentences * words) /
extractedCount).toFixed(1) : "N/A"
        };

        return reconstructionQuality;
    }

    function analyzeText(text) {
```

```
        if (!text.trim()) return [];

        const findings = [];
        const foundPatterns = new Set();

        ocsPatterns.forEach(pattern => {
            if (!activeLayers[pattern.layer]) return;

            let match;
            const regex = new RegExp(pattern.regex);

            while ((match = regex.exec(text)) !== null) {
                const key = `${pattern.name}:${match[0]}:${match.index}`;

                if (!foundPatterns.has(key)) {
                    foundPatterns.add(key);
                    findings.push({
                        type: pattern.type,
                        name: pattern.name,
                        message: pattern.message(match[0]),
                        context: match[0],
                        layer: pattern.layer,
                        severity: pattern.severity,
                        position: match.index,
                        mechanism: pattern.mechanism,
                        propagation: pattern.propagation,
                        metrological: pattern.metrological
                    });
                }
            }
        });

        // Structural analysis
        const sentences = text.split(/[.!?]+/).filter(s => s.trim());

        sentences.forEach((sentence, idx) => {
            const trimmed = sentence.trim();

            if (trimmed.length > 250) {
                findings.push({
                    type: "warning",
                    name: "Complex Sentence Structure",
                    message: "Long sentence (250+ chars) may obscure dependencies and
compress structure",
                    context: trimmed.substring(0, 40) + "...",
                    layer: "phaseC",
```

```
                    severity: 5,
                    propagation: "Dependencies may be hidden within",
                    metrological: { confidence: "High", observability: "Direct", measurement:
"Determinable from source" },
                    position: 0
                });
            }
        });

        previousAnalysis = findings;
        return findings.sort((a, b) => {
            const typeOrder = { critical: 0, error: 1, warning: 2, info: 3, success: 4 };
            if (typeOrder[a.type] !== typeOrder[b.type]) {
                return typeOrder[a.type] - typeOrder[b.type];
            }
            return b.severity - a.severity;
        });
    }

    function renderFindings(findings) {
        const container = document.getElementById("findings");

        if (findings.length === 0) {
            container.innerHTML = '<div class="empty-state">✓ Structural completeness
verified (or text is empty)</div>';
            updateStats(findings);
            renderBidirectional("");
            return;
        }

        const text = document.getElementById("textInput").value;
        const criticalCount = findings.filter(f => f.type === "critical").length;
        const warningCount = findings.filter(f => f.type === "warning").length;
        const infoCount = findings.filter(f => f.type === "info").length;

        container.innerHTML = findings.map(finding => `
            <div class="finding ${finding.type}">
                <div class="finding-type">
                    <span>${finding.name}</span>
                    <span class="severity">${finding.type.toUpperCase()} | SEV: $
{finding.severity}/10</span>
                </div>
                <div class="finding-text">${finding.message}</div>
                <div class="finding-detail">Context: "${finding.context}"</div>
```

```
                ${finding.mechanism ? `<div class="finding-mechanism">⚙️ $
{finding.mechanism}</div>` : ''}
                ${finding.metrological ? `<div class="finding-observability">📊 Confidence:
${finding.metrological.confidence} | Observability: ${finding.metrological.observability} |
Measurement: ${finding.metrological.measurement}</div>` : ''}
                <div class="finding-propagation">↓ Propagation: ${finding.propagation}</
div>
                <div>
                    <span class="badge layer">${finding.layer}</span>
                    <span class="badge metrological">METROLOGICAL</span>
                </div>
            </div>
        `).join("");

        updateStats(findings);
        renderBidirectional(text);
    }

    function renderBidirectional(text) {
        const section = document.getElementById("bidirectionalSection");

        if (!text.trim()) {
            section.innerHTML = '';
            return;
        }

        const findings = previousAnalysis || [];
        const quality = bidirectionalValidation(text, findings);
        const collapses = detectSynonymCollapse(text);

        let html = '<div class="bidirectional-check"><h4>↔️ Bidirectional Validation &
Anti-Synonym Check</h4>';

        html += '<div style="margin-bottom: 6px;"><strong>Reconstruction Quality:</
strong></div>';
        html += `<div class="bi-item">Sentences: ${quality.sentencePreservation} |
Words: ${quality.wordDensity} | Compression Ratio: ${quality.compressionRisk}</div>`;
        html += `<div class="bi-item">Components Detected: $
{quality.componentExtraction} | Mechanisms: ${quality.mechanismPreservation} |
Uncertainties: ${quality.uncertaintyMarking}</div>`;

        if (collapses.length > 0) {
            html += '<div style="margin-top: 8px;"><strong>⚠️ Synonym Collapse
Risks:</strong></div>';
```

```
            collapses.forEach(collapse => {
                html += `<div class="synonym-group"><h5>${collapse.group}</h5>`;
                html += `<div class="synonym-items">Items: ${collapse.items.join(", ")}</
div>`;
                html += `<div class="synonym-items">Distinguishing factors: $
{collapse.differences}</div></div>`;
            });
        } else {
            html += '<div style="margin-top: 8px; color: #6bff6b;">✓ No synonym collapse
detected</div>';
        }

        html += '</div>';
        section.innerHTML = html;
    }

    function updateStats(findings) {
        const stats = document.getElementById("stats");
        const critical = findings.filter(f => f.type === "critical").length;
        const warnings = findings.filter(f => f.type === "warning").length;
        const info = findings.filter(f => f.type === "info").length;
        const total = findings.length;

        stats.innerHTML = `
            <div class="stat-item">
                <div class="stat-value">${critical}</div>
                <div class="stat-label">Critical</div>
            </div>
            <div class="stat-item">
                <div class="stat-value">${warnings}</div>
                <div class="stat-label">Warnings</div>
            </div>
            <div class="stat-item">
                <div class="stat-value">${info}</div>
                <div class="stat-label">Info</div>
            </div>
            <div class="stat-item">
                <div class="stat-value">${total}</div>
                <div class="stat-label">Total</div>
            </div>
        `;
    }

    document.getElementById("checkBtn").addEventListener("click", () => {
        const text = document.getElementById("textInput").value;
        const findings = analyzeText(text);
```

```
            renderFindings(findings);
        });

        document.getElementById("refineBtn").addEventListener("click", () => {
            if (!previousAnalysis || previousAnalysis.length === 0) {
                alert("Run analysis first to enable refinement protocol");
                return;
            }

            const refineText = `
D₁ - LINE-BY-LINE INGESTION: Reading findings in exact order...
${previousAnalysis.slice(0, 3).map(f => `  • [${f.layer}] ${f.name}: REQUIRES
REVISION`).join('\n')}

D₂ - REVISION LEDGER UPDATE:
  Source: Current analysis
  Invalidation status: FLAGGED FOR REVISION
  Carry-forward: Maintain metrological grounding

D₃ - CONFLICT DETECTION:
  Status: Ready for user intervention

D₄ - HIERARCHICAL INTEGRATION:
  New findings reinserted into phase layers...
  Structural revalidation pending...

D₅ - STRUCTURAL REVALIDATION:
  Coverage audit complete
  Reverse reconstruction test: READY
            `;

            alert(refineText);
        });

        document.getElementById("clearBtn").addEventListener("click", () => {
            document.getElementById("textInput").value = "";
            document.getElementById("findings").innerHTML = '<div class="empty-
state">Enter text and click "Analyze" to begin</div>';
            document.getElementById("stats").innerHTML = '';
            document.getElementById("bidirectionalSection").innerHTML = '';
            previousAnalysis = null;
        });

        document.querySelectorAll(".layer-btn").forEach(btn => {
            btn.addEventListener("click", (e) => {
                const layer = e.target.dataset.layer;
```

```
            activeLayers[layer] = !activeLayers[layer];
            e.target.classList.toggle("active");

            const text = document.getElementById("textInput").value;
            if (text.trim()) {
                const findings = analyzeText(text);
                renderFindings(findings);
            }
        });
    });

    document.getElementById("textInput").addEventListener("keyup", (e) => {
        if (e.ctrlKey && e.key === "Enter") {
            document.getElementById("checkBtn").click();
        }
    });
  </script>
</body>
</html>
```